

CSCE 627 THEORY OF COMPUTABILITY · MODULE 12 · LECTURE NOTES

The Engineering Response

What to do about an undecidable problem.

MODULE OUTCOMES

- Name the four strategies and choose between them.
- Explain abstract interpretation and soundness by construction.
- Explain the certificate strategy and its trade.
- Design an analysis with a stated soundness position.
- Explain why the choice is a product decision.

1 The four strategies

Strategy	What it gives up	Examples
Approximate	Precision.	Abstract interpretation, type systems, dataflow analysis. See the callout.
Restrict the input language	Expressiveness.	Total functional languages, non-Turing-complete configuration formats, decidable logical fragments (§2).
Bound a resource	Coverage.	Bounded model checking, Ethereum's gas, timeouts, loop unrolling limits (Module 06 §2).
Demand a certificate	Automation.	Proof-carrying code, dependent types, Rust lifetimes, proof assistants (§3).

Each gives up something different, which is why they compose rather than compete — a real tool typically uses two or three at once (a bounded abstract interpretation over a restricted language, with programmer annotations where it gets stuck), and stating which is what makes its output interpretable.

APPROXIMATION CAN BE SOUND BY CONSTRUCTION

Instead of computing exact values, compute a deliberate over-approximation. Instead of 'x = 7', compute 'x is in [0, 10]' or 'x is positive' — less information, and obtainable without running the program.

Design the abstract domain and its operations so that the abstract result always *contains* the concrete one — then **soundness is a property of the construction rather than something to test for**, which is the central idea of abstract interpretation and is what makes it suitable for safety-critical work.

And the analysis terminates because the abstract domain has finite height, or because a *widening* operator forces convergence after finitely many steps. **Termination is built in as well.**

So you get sound and terminating, and lose precision — which is Module 08 §2's pick-two-of-three, with the loss placed somewhere you can measure, tune, and trade against cost. **Putting the unavoidable loss in a tunable place is the engineering achievement**, not the soundness itself.

```
CONCRETE: x = 5; y = 3; z = x + y;      -> z = 8
```

```
ABSTRACT, with the SIGN domain:
```

```
  x : positive
  y : positive
  z : positive + positive = positive
```

The abstract result is CORRECT and LESS INFORMATIVE, and it was computed WITHOUT running the program.

THE DESIGN OBLIGATIONS

```
a DOMAIN      signs, intervals, octagons, polyhedra
a TRANSFER FUNCTION per operation, which must
    over-approximate the concrete one
a JOIN for merging control-flow branches
    positive join negative = unknown
and WIDENING for loops, to force convergence
    [0,1] then [0,2] then [0,3] ... widen to
    [0, infinity]
```

PRECISION IS A DIAL: signs are cheap and coarse; intervals better; octagons (constraints of the form $x - y \leq c$) better again; convex polyhedra precise and expensive.

ASTREE verified the ABSENCE OF RUNTIME ERRORS in Airbus flight control code using exactly this. Sound, terminating, and it required engineers to select and tune domains per module -- which is what the approach costs in practice.

2 Restriction and bounding

- **Total functional languages.** Agda, Idris, Coq's terminating fragment. **Termination is guaranteed by the type system, and some computable total functions cannot be written** (Module 09 §1's theorem, which is exactly the price).

- **Non-Turing-complete configuration languages.** Dhall, Starlark, and similar. **Deliberate weakness** (Module 11 §2), chosen so that configurations can be analysed, diffed, and bounded.
- **Decidable logical fragments.** Presburger arithmetic, linear real arithmetic, bit-vectors, arrays, effectively-propositional formulas — **which is how SMT solvers are assembled** (Module 10 §4), and the engineering consists in staying inside them.
- **Structured programming. Bounded loops and the absence of arbitrary goto make reaching-definition and dominance analyses straightforward** — which is a retrospective theoretical justification for what was argued on stylistic grounds in 1968.
- **And type systems themselves**, which restrict what can be expressed precisely in order to decide something about what remains. **Restriction is the most underused of the four strategies**, because **it has to be chosen at design time** and cannot be retrofitted — **and it is the only one that makes the problem genuinely go away** rather than managing it.

BOUNDING IS THE EASIEST AND THE MOST HONEST

Unroll loops k times; explore to depth d ; stop after t seconds. The bounded question is decidable, always, by Module 06 §2's bounded-property observation — there is no cleverness required.

And the resulting claim is clean *provided you state the bound*: 'no error reachable within 20 transitions' is a complete, checkable, and genuinely useful statement about a system.

The failure mode is reporting the bounded result as though it were unbounded — which Module 08 §3 flagged, and which is a **documentation failure rather than a technical one**, and therefore entirely avoidable.

And it composes with the others: bounded abstract interpretation, bounded search over a restricted language, bounded checking with programmer annotations where the bound bites — **real tools stack them**, and the stacking is where the practical capability comes from.

3 Certificates

IF FINDING THE PROOF IS UNDECIDABLE, CHECKING ONE IS NOT

Module 04 §2's recognisability result means a proof can be *checked* mechanically even when it cannot be *found* — the asymmetry between Σ_1 and decidable, put to work.

So demand the proof as an input. Proof-carrying code, verified compilers' translation certificates, dependently typed programs, and Rust's lifetime annotations are all instances of this — the programmer or an upstream tool supplies the witness, and the checker verifies it.

The cost is that a human or a separate tool must produce the certificate — which is real work, and is why formal verification is expensive rather than why it is rare.

And the gain is a sound *and* complete checker — the only position in Module 08 §2's table that achieves both, because the undecidable search has been moved outside the tool. That is the whole trade, and it is worth seeing clearly: you have not escaped the theorem, you have relocated the work to somewhere a human can apply judgement.

Approach	The certificate is	Produced by
Rust ownership and borrowing	Lifetime annotations, mostly inferred.	The programmer, with substantial inference — which is why Rust feels like an ordinary language most of the time and like a proof assistant occasionally.
Dependent types	The type itself is the proof.	The programmer, and this is the whole development effort in Agda or Idris.
Proof-carrying code	A machine-checkable proof shipped with the binary.	A certifying compiler, or a separate prover.
Verified compilation (CompCert)	A proof that this output matches this input's semantics.	The compiler, once per compilation — or, in CompCert's case, a once-and-for-all proof about the compiler.
SAT unsatisfiability proofs	A DRAT refutation.	The solver itself (CSCE 625 Module 08 §2).

The solver-produced certificate is the best case: the tool does all the work and emits something an independent, much simpler checker can verify — so you need not trust the complicated tool at all, only the small checker. That is the real value of certificates, and it is why unsatisfiability proofs changed how SAT solvers are used in verification.

4 Choosing, and designing

WHICH STRATEGY DEPENDS ON WHAT A WRONG ANSWER COSTS

A missed bug is catastrophic — avionics, medical devices, cryptographic implementations? Be sound, accept the false positives, and staff the triage. Astrée and SPARK are in this position, and their users accept a high alarm rate because the alternative is unacceptable.

Developers will ignore a noisy tool entirely? Be near-complete, miss bugs deliberately, and keep the signal-to-noise ratio high enough that the tool stays in the build. **Infer's team state this explicitly,** and it is a correct engineering judgement rather than a compromise: a sound tool nobody runs finds nothing.

The guarantee itself is the product? Demand certificates (§3) and accept the cost in developer effort — which is the seL4 and CompCert position.

And in every case: state the position in the documentation. A tool whose users do not know whether 'no issues found' means 'proved safe' or 'found nothing' is harmful regardless of its technical quality, because the users will assume whichever is more convenient. **This is the one failure in the module that is purely a writing failure.**

- **1 · Prove the problem undecidable,** or recognise it immediately by Rice's theorem (Module 06 §2). **Ten minutes, and it prevents a month of searching for an exact algorithm.**
- **2 · Decide what a wrong answer costs, in each direction separately. A false positive and a false negative almost never cost the same,** and the asymmetry determines everything downstream.
- **3 · Choose which of sound, complete, and terminating to give up, and write it down where users will see it.**
- **4 · Choose strategies from §1's table,** usually two or three in combination.
- **5 · Then build it — and construct your own false positive and false negative before anyone else does** (Module 08 §4). **Steps 1 through 3 take an hour and determine everything after them,** which makes them **the highest-value hour in building an analysis tool** and the one most often skipped.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Cousot & Cousot — Abstract Interpretation papers (free)	§1's framework from its originators, including the widening operator and the soundness-by-construction argument.
Blanchet et al. — A Static Analyzer for Large Safety-Critical Software (Astrée) (free)	§1's industrial result, including what the domain tuning actually involved.
Necula & Lee — Proof-Carrying Code (free)	§3's approach, and the clearest statement of the check-versus-find asymmetry.

Resource	What it covers for this module
Facebook Infer — the design documentation (free)	§4's near-complete position, stated explicitly by its authors — a rare and commendable piece of honest tool documentation.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a sign-domain abstract interpreter** for a small imperative language.
- 2 **Extend it to intervals** and report where the precision improves.
- 3 **Add a widening operator** and confirm loops now converge.
- 4 **Construct a program where the sign domain loses the answer** and the interval domain keeps it.
- 5 **Verify your transfer functions over-approximate** by testing against concrete execution.
- 6 **Take an analysis problem and apply all four strategies**, sketching each.
- 7 **Find a configuration language that is deliberately weak** and identify what it gave up.
- 8 **Run a bounded checker at two bounds** and write the honest claim for each.
- 9 **Get a DRAT proof from a SAT solver** and verify it with an independent checker.
- 10 **Design an analysis for a problem you care about**, following §4's five steps, and write down steps 1 to 3 before coding.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four strategies and what each gives up.
- 2 Why do they compose?
- 3 Explain soundness by construction in abstract interpretation.
- 4 Name the four design obligations of an abstract domain.
- 5 Why is precision the right place to put the loss?
- 6 Name five restriction examples and why restriction is underused.
- 7 What makes bounding honest, and what is its failure mode?
- 8 Why does a certificate give both soundness and completeness?
- 9 Why is a solver-produced certificate the best case?
- 10 Give the five design steps and say which three matter most.